

Eng 4.2. OWASP API Security Top 10 - 2

Vulnerability VI — Mass Assignment

Mass assignment reflects a scenario where client-side data is automatically bound with server-side objects or class variables. However, hackers exploit the feature by first understanding the application's business logic and sending specially crafted data to the server, acquiring administrative access or inserting tampered data. This functionality is widely exploited in the latest frameworks like Laravel, Code Ignitor etc. Consider a user's profiles dashboard where users can update their profile like associated email, name, address etc. The username of the user is a read-only attribute and cannot be changed; however, a malicious actor can edit the username and submit the form. If necessary filtration is not enabled on the server side (model), it will simply insert/update the data in the database.

Likely Impact: The attack may result in data tampering and privilege escalation from a regular user to an administrator.

Practical Example

- Open the VM. You will find that the Chrome browser and Talend API Tester application are running automatically, which we will be using for debugging the API endpoints.
- Bob has been assigned to develop a signup API endpoint `/apirule6/user` that will take a name, username and password as input parameters (POST). The user's table has a `credit column` with a default value of `50`. Users will upgrade their membership to have a larger credit value.
- Bob has successfully designed the form and used the mass assignment feature in Laravel to store all the incoming data from the client side to the database (as shown below).

DRAFT

VULNERABLE

Save as

METHOD

POST

SCHEME :// HOST [:" PORT] [PATH ["?" QUERY]]

http://localhost:80/MHT/apirule6/user

length: 37 byte(s)

Send

QUERY PARAMETERS

HEADERS

Form

☒ Content-Type

:

application/x-www-form-ur

×

+ Add header

+ Add authorization

BODY

Form

☒ name

[Text]

=

Bob

×

☒ username

[Text]

=

bob_mht

×

☒ password

[Text]

=

#ge*byA@35U6

×

☒ credit

[Text]

=

110

×

+ Add form parameter

☒ application/x-www-form-urlencoded

Response

Cache Detected - Elapsed Time: 529ms

201 Created

HEADERS

pretty

Host: localhost

Date: Tue, 02 Aug 2022 09:41:43 GMT -1s

Connection: close

X-Powered-By: PHP/8.0.9

Cache-Control: no-cache, private

Date: Tue, 02 Aug 2022 09:41:43 GMT -1s

Content-Type: application/json

COMPLETE REQUEST HEADERS

BODY

pretty

```
{
  name: "Bob",
  username: "bob_mht",
  credit: "110",
  id: 3
}
```

length: 57 bytes

The solution to the problem is pretty simple. Bob must ensure necessary filtering on the server side (apirule6/user_s) and ensure that the default value of credit should be inserted as 50, even if more than 50 is received from the client side (as shown below).

SECURE

METHOD: POST SCHEME://HOST [:" PORT] [PATH ["?" QUERY]] length: 39 byte(s)

http://localhost:80/MHT/apirule6/user_s Send

QUERY PARAMETERS

HEADERS + Form

☒ Content-Type : application/x-www-form-urlencoded ×

+ Add header + Add authorization

BODY + Form

☒ name [Text] = Bob ×

☒ username [Text] = bob_mht ×

☒ password [Text] = #ge*byA@35U6 ×

☒ credit [Text] = 110 ×

+ Add form parameter ☒ application/x-www-form-urlencoded ×

Response Cache Detected - Elapsed Time: 605ms

201 Created

HEADERS pretty

Host: localhost

Date: Tue, 02 Aug 2022 10:03:53 GMT

Connection: close

X-Powered-By: PHP/8.0.9

Cache-Control: no-cache, private

Date: Tue, 02 Aug 2022 10:03:53 GMT

Content-Type: application/json

BODY pretty

```
{
  name: "Bob",
  username: "bob_mht",
  credit: 50,
  id: 4
}
```

length: 54 bytes

Mitigation Measures

- Before using any framework, one must study how the backend insertions and updates are carried out. In the Laravel framework, fillable and guarded (opens in new tab) arrays mitigate the above-mentioned scenarios.
- Avoid using functions that bind an input from a client to code variables automatically.
- Allowlist those properties only that need to get updated from the client side.

Answer the questions below

Is it a good practice to blindly insert/update user-provided data in the database (yea/nay)?

Answer: **Nay**

Using /apirule6/user_s, insert a record in the database using the credit value as 1000.

What would be the returned credit value after performing Question#2?

Request:

METHOD: POST
 SCHEME // HOST [PORT] [PATH] [QUERY]
 http://127.0.0.1/MHT/apirule6/user_s

QUERY PARAMETERS: (empty)

HEADERS: Content-Type: application/x-www-form-urlencoded

BODY (Form): name=Bob, credit=1000

Response:

201 Created

HEADERS: Host: 127.0.0.1, Date: Sun, 17 Sep 2023 03:44:53 GMT, Connection: close, X-Powered-By: PHP/7.4.29, Cache-Control: no-cache, private, Date: Sun, 17 Sep 2023 03:44:53 GMT, Content-Type: application/json

BODY (pretty): { name: "Bob", credit: 50, id: 6 }

Answer: **50**

Vulnerability VII — Security Misconfiguration

Security misconfiguration depicts an implementation of incorrect and poorly configured security controls that put the security of the whole API at stake. Several factors can result in security misconfiguration, including improper/incomplete default configuration, publically accessible cloud storage, Cross-Origin Resource Sharing (CORS) (opens in new tab), and error messages displayed with sensitive data. Intruders can take advantage of these misconfigurations to perform detailed reconnaissance and get unauthorised access to the system.

Security misconfigurations are usually detected by vulnerability scanners or auditing tools and thus can be curtailed at the initial level. API documentation, a list of endpoints, error logs etc., must not be publically accessible to ensure safety against security misconfigurations. Typically, companies deploy security controls like web application firewalls, which are not configured to block undesired requests and attacks.

Likely Impact: Security misconfiguration can give intruders complete knowledge of API components. Firstly, it allows intruders to bypass security mechanisms. Stack trace or other detailed errors can provide the malicious actor access to confidential data and essential system details, further aiding the intruder in profiling the system and gaining entry.

Practical Example

- Continue to use the Chrome browser and Talend API Tester for debugging in the VM.
- The company MHT is facing serious server availability issues. Therefore, they assigned Bob to develop an API endpoint `/apirule7/ping_v` (GET) that will share details regarding server health and status.
- Bob successfully designed the endpoint; however, he forgot to implement any error handling to avoid any information leakage.

The screenshot shows the Talend API Tester interface. At the top, there is a red banner that says "VULNERABLE" and a button that says "Ask Echo". Below this, the request is configured with the method "GET" and the URL "http://localhost:80/MHT/apirule7/ping_v". The response is shown below the request, with a status of "200 OK". The response body is displayed in a "pretty" format, showing a stack trace from a Laravel application. The stack trace includes the following lines:

```
string(6541) "#0
C:\xampp\htdocs\MHT\MHT\MHT\vendor\laravel\framework\src\Illuminate\R
App\Http\Controllers\API7UsersController-
auth_v(Object(Illuminate\Http\Request))
#1
C:\xampp\htdocs\MHT\MHT\MHT\vendor\laravel\framework\src\Illuminate\R
lluminate\Routing\Controller->callAction('auth_v', Array)
#2
C:\xampp\htdocs\MHT\MHT\MHT\vendor\laravel\framework\src\Illuminate\R
lluminate\Routing\ControllerDispatcher-
>dispatch(Object(Illuminate\Routing\Route),
```

- The solution to the issue is pretty simple. Bob will create an API endpoint `/apirule7/ping_s` that will carry out error handling and only share desired

information with the user (as shown below).

The screenshot shows a web client interface with a green 'SECURE' banner at the top. The 'METHOD' dropdown is set to 'GET' and the 'URL' field contains 'http://localhost:80/MHT/apirule7/ping_s'. The 'Send' button is visible. Below the URL bar, the 'QUERY PARAMETERS' section is empty. The 'HEADERS' section is on the left, and the 'BODY' section is on the right, showing a message: 'XHR does not allow payloads for GET request.' Below the main interface, the 'Response' section is displayed, showing a '500 Internal Server Error' status. The response body is a JSON object: `{"success": "false", "msg": "Network Server @ 2 - Malfunctioned - Error ID #1401. Please contact administrator at support@mht.com for further queries."}`. The response headers are also visible, including 'Host: localhost', 'Date: Tue, 02 Aug 2022 12:33:04 GMT', 'Connection: close', 'X-Powered-By: PHP/8.0.9', 'Cache-Control: no-cache, private', and 'Content-Type: application/json'.

Mitigation Measures

- Limit access to the administrative interfaces for authorised users and disable them for other users.
- Disable default usernames and passwords for public-facing devices (routers, Web Application etc.).
- Disable directory listing and set proper permissions for every file and folder.
- Remove unnecessary pieces of code snippets, error logs etc. and turn off debugging while the code is in production.

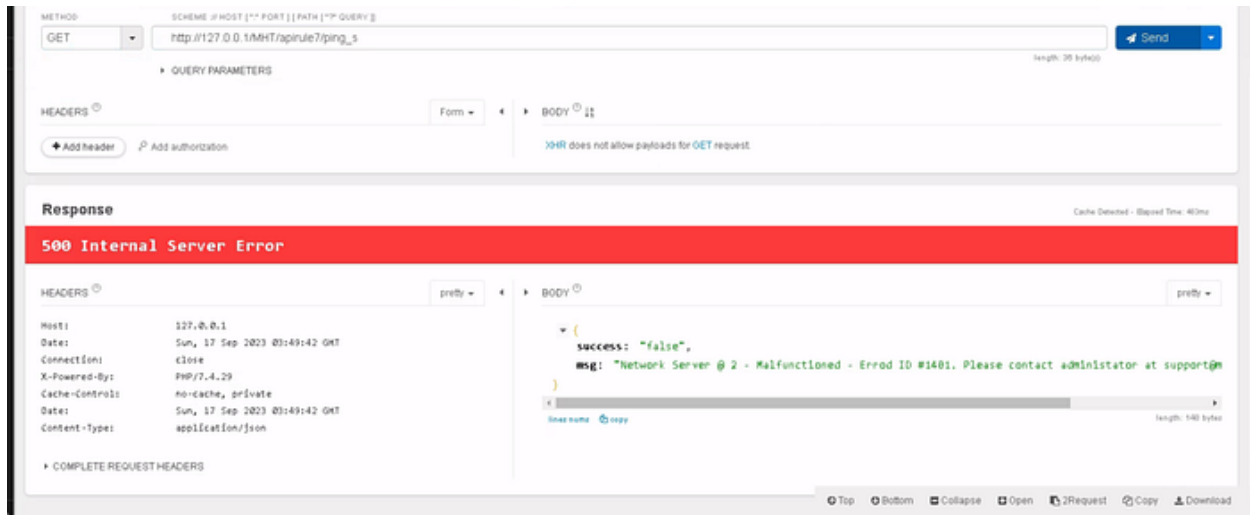
Answer the questions below

Is it an excellent approach to show error logs from the stack trace to general visitors (yea/nay)?

Answer: **Nay**

Try to use the API call /apirule7/ping_s in the attached VM.

What is the HTTP response code?



Answer: **500**

What is the Error ID number in the HTTP response message?

Answer: **1401**

Vulnerability VIII — Injection

Injection attacks are probably among the oldest API /web-based attacks and are still being carried out by hackers on real-world applications. Injection flaws occur when user input is **not filtered and is directly processed by an API** ; thus enabling the attacker to perform unintended API actions without authorisation. An injection may come from Structure Query Language (SQL), operating system (OS) commands, Extensible Markup Language (XML) etc. Nowadays, frameworks offer functionality to protect against this attack through automatic sanitisation of data; however, applications built in custom frameworks like core are still susceptible to such attacks.

Likely Impact: Injection flaws may lead to **information disclosure, data loss, , and complete account takeover**. The successful injection attacks may also cause the intruders to access

the sensitive data or even create new functionality and perform remote code execution.

Practical Example

- Continue to use the Chrome browser and Talend API Tester for debugging in the VM.
- A few users of company MHT reported that their account password had changed, and they could not further log in to their original account. Consequently, the dev team found that Bob had developed a vulnerable login API endpoint `/apirule8/user/login_v` that is not filtering user input.
- A malicious attacker requires the username of the target, and for the password, they can use the payload `' OR 1=1--'` and get an authorisation key for any account (as shown below).

The screenshot displays the Talend API Tester interface. At the top, a red banner reads "VULNERABLE". The request is configured as follows:

- METHOD:** POST
- URL:** `http://localhost:80/MHT/apirule8/user/login_v` (length: 45 byte(s))
- QUERY PARAMETERS:** None
- BODY:** Form data with two parameters:
 - password:** `' OR 1=1--'`
 - username:** `admin`
- Headers:** `application/x-www-form-urlencoded`

The response is shown below the request:

- Status:** 200 OK
- Headers:**
 - Host: localhost
 - Date: Wed, 03 Aug 2022 18:08:27 GMT
 - Connection: close
 - X-Powered-By: PHP/8.0.9
 - Cache-Control: no-cache, private
 - Date: Wed, 03 Aug 2022 18:08:27 GMT
 - Content-Type: application/json
- Body:**

```
{
  "success": "true",
  "authkey": "oWsZ8vNnUECjCAiZVJH0zsNsNH08zWRZ"
}
```

 (length: 63 bytes)

Bob immediately realised his mistake; he updated the API endpoint to `/apirule8/user/login_s` and used parameterised queries and built-in filters of

Laravel to sanitise user input. As a result, all malicious payloads on username and password parameters were effectively mitigated (as shown below)

The screenshot displays a web security tool interface. At the top, a green bar indicates the connection is **SECURE**. The request method is **POST** to the URL **http://localhost:80/MHT/apirule8/user/login_s**. The request body is in **Form** mode with the following parameters:

- password**: `'OR 1=1--'`
- username**: `admin`

The response status is **200 OK** (indicated by a red bar). The response body is a JSON object:

```
{  "success": "false",  "cause": "IncorrectUsernameOrPassword"}
```

Mitigation Measures

- Ensure to use a well-known library for client-side input validation.
- If a framework is not used, all client-provided data must be validated first and then filtered and sanitised.
- Add necessary security rules to the Web Application Firewall (WAF). Most of the time, injection flaws can be mitigated at the network level.
- Make use of built-in filters in frameworks like Laravel, Code Ignitor etc., to validate and filter data.

Answer the questions below

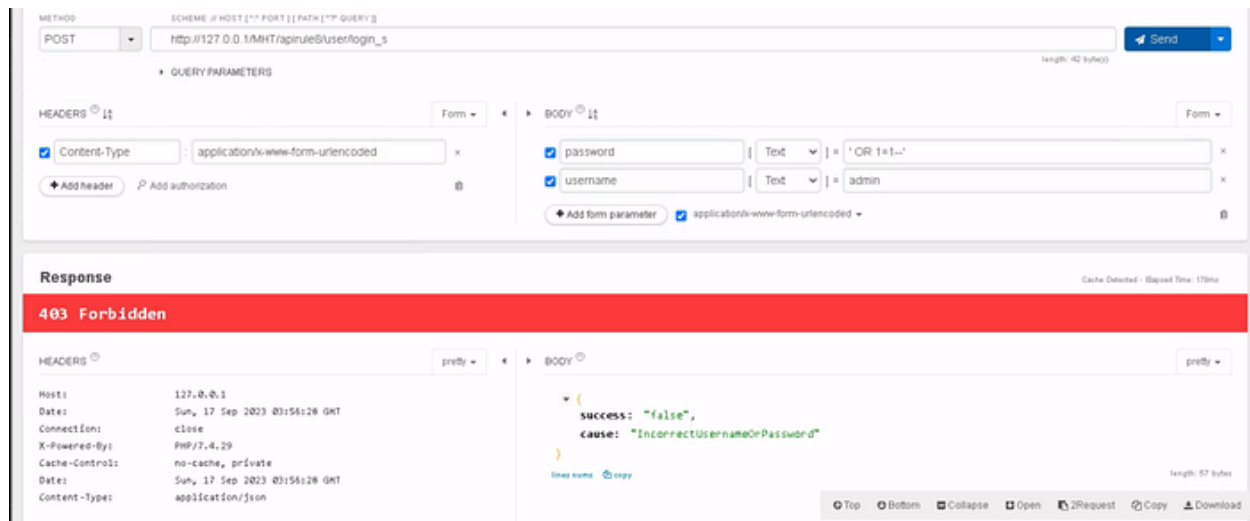
Can injection attacks be carried out to extract data from the database (yea/nay)?

Answer: **Yea**

Can injection attacks result in remote code execution (yea/nay)?

Answer: **Yea**

What is the HTTP response code if a user enters an invalid username or password?



Answer: **403**

Vulnerability IX — Improper Assets Management

Inappropriate Asset Management refers to a scenario where we have **two versions of an API available in our system**; let's name them APIv1 and APIv2.

Everything is wholly switched to

APIv2, but the previous version, APIv1, has not been deleted yet. Considering this, one might easily guess that the older version of the API i.e., APIv1, doesn't have the updated or the latest security features. Plenty of other obsolete features of APIv1 make it possible to find vulnerable scenarios, which may lead to data leakage and server takeover via a shared database amongst API versions. It is essentially about not properly tracking API endpoints. The potential reasons could be incomplete documentation or absence of compliance with the Software Development Life Cycle. A properly maintained, up-to-date API inventory and proper documentation are more critical than hardware-based security control for an organisation.

Likely Impact: The older or the **unpatched API versions** can allow the intruders to get unauthorised access to confidential data or even complete control of the system.

Practical Example

- Continue to use the Chrome browser and Talend API Tester for debugging in the VM.
- During API development, the company MHT has developed different API versions like v1 and v2. The company ensured to use the latest versions and API calls but forgot to remove the old version from the server.
- Consequently, it was found that old APIS calls like `apirule9/v1/user/login` return more information like balance, address etc., against the user (as shown below).

The screenshot shows the Talend API Tester interface. At the top, a red banner reads "OLDER VERSION". The request is a POST to `http://localhost:80/MHT/apirule9/v1/user/login`. The body is a form with parameters `username=alice` and `password=###@##`. The response is a 200 OK status with a JSON body: `{id: 1, username: "alice", Balance: [REDACTED], country: [REDACTED]}`. The response headers show it's from localhost, dated Wed, 03 Aug 2022 18:44:46 GMT, with content-type application/json.

Bob being the developer of the endpoint, realised that he must immediately deactivate old and unused assets so that users can only access limited and desired information from the new endpoint `/apirule9/v2/user/login` (as shown below)

The screenshot shows a web client interface with a green 'SECURE' banner at the top. The request is a POST to 'http://localhost:80/MHT/apirule9/v2/user/login'. The body is a form with 'username' set to 'alice' and 'password' set to '###!@#!'. The response is a 200 OK with a JSON body: {'id': 1, 'username': 'alice'}. The interface includes a 'Send' button, a 'Save as' dropdown, and a 'Response' section with 'HEADERS' and 'BODY' tabs. The 'BODY' tab is selected, showing the JSON response. The 'username' field in the request body is highlighted with a red box, and the 'id' and 'username' fields in the response body are also highlighted with a red box.

Request:

METHOD: POST
URL: http://localhost:80/MHT/apirule9/v2/user/login
QUERY PARAMETERS: (empty)
BODY: Form
username: [Text] = alice
password: [Text] = ###!@#!
Content-Type: application/x-www-form-urlencoded

Response:

200 OK
Cache Detected - Elapsed Time: 341ms
HEADERS: Host: localhost, Date: Wed, 03 Aug 2022 18:49:13 GMT -1s, Connection: close, X-Powered-By: PHP/8.0.9, Cache-Control: no-cache, private, Date: Wed, 03 Aug 2022 18:49:13 GMT -1s, Content-Type: application/json
BODY: {id: 1, username: "alice"}
length: 27 bytes

Mitigation Measures

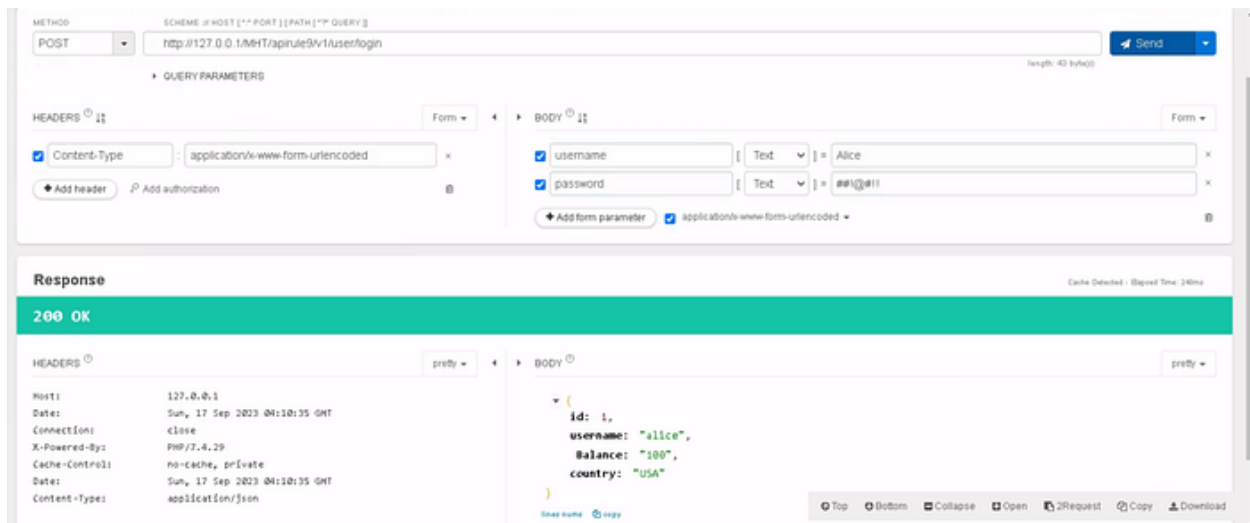
- Access to previously developed sensitive and deprecated API calls must be blocked at the network level.
- APIs developed for R&D, QA, production etc., must be segregated and hosted on separate servers.
- Ensure documentation of all API aspects, including authentication, redirects, errors, CORS policy, and rate limiting.
- Adopt open standards to generate documentation automatically.

Answer the questions below

Is it good practice to host all APIs on the same server (yea/nay)?

Answer: **Nay**

Make an API call to /apirule9/v1/user/login using the username "**Alice**" and password "**###!@#!**".



What is the amount of balance associated with user Alice?

Answer: **100**

What is the country of the user Alice?

Answer: **USA**

Vulnerability X — Insufficient Logging & Monitoring

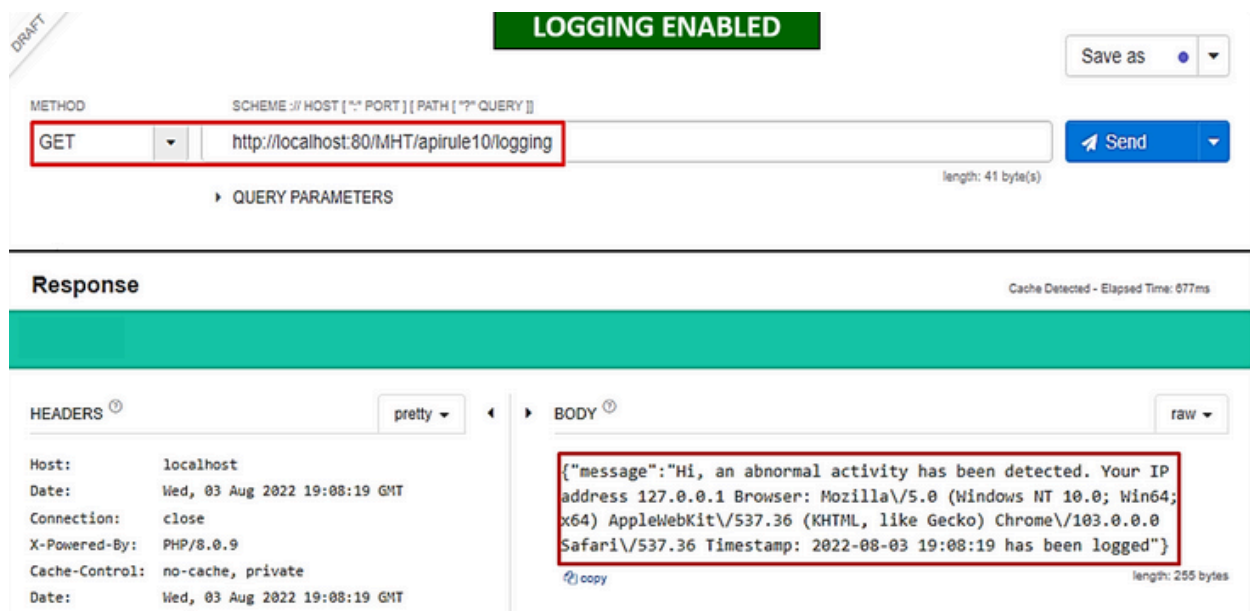
Insufficient logging & monitoring reflects a scenario when an attacker conducts malicious activity on your server; however, when you try to track the hacker, **there is not enough evidence available due to the absence of logging and monitoring mechanisms**. Several organisations only focus on infrastructure logging like network events or server logging but lack API logging and monitoring. Information like the visitor's IP address, endpoints accessed, input data etc., along with a timestamp, enables the identification of threat attack patterns. If logging mechanisms are not

in place, it would be challenging to identify the attacker and their details. Nowadays, the latest web frameworks can automatically log requests at different levels like error, debug, info etc. These errors can be logged in a database or file or even passed to a SIEM solution for detailed analysis.

Likely Impact: Inability to identify attacker or hacker behind the attack.

Practical Example

- Continue to use the Chrome browser and Talend API Tester for debugging in the VM.
- In the past, the company MHT has been susceptible to multiple attacks, and the exact culprit behind the attacks could not be identified.
Therefore, Bob was assigned to make an API endpoint `/apirule10/logging` (GET) that will log users' metadata (IP address, browser version etc.) and save it in the database as well (as shown below).



Mitigation Measures

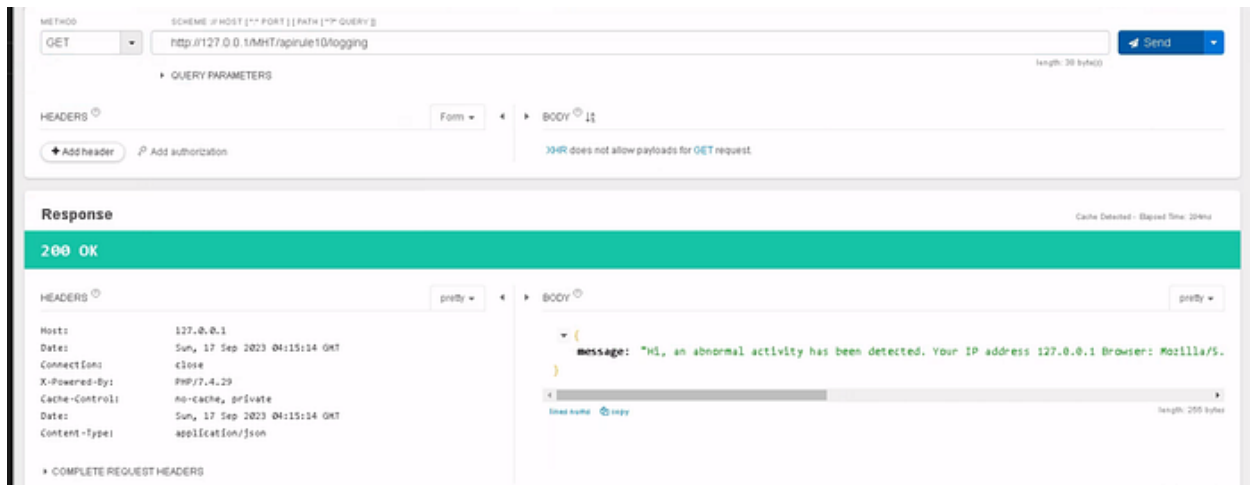
- Ensure use of the Security Information and Event Management (SIEM) system for log management.
- Keep track of all denied accesses, failed authentication attempts, and input validation errors, using a format imported by SIEM and enough detail to identify the intruder.
- Handle logs as sensitive data and ensure their integrity at rest and transit. Moreover, implement custom alerts to detect suspicious activities as well.

Answer the questions below

Should the API logs be publically accessible so that the attacker must know they are being logged (yea/nay)?

Answer: **Nay**

What is the HTTP response code in case of successful logging of user information?



Answer: **200**

Conclusion

Most commonly, API systems are hacked because of failure in authorisation and authentication mechanisms and security misconfigurations. API Developers must **safeguard APIs in line with best cyber security practices**. The modules like sign-in, role-based access, user profile setting etc., must be given more importance as malicious actors tend to target known endpoints for gaining access to the system.